

CHAPTER 03

MCP Architecture, Primitives and Protocol



LEARN



BUILD

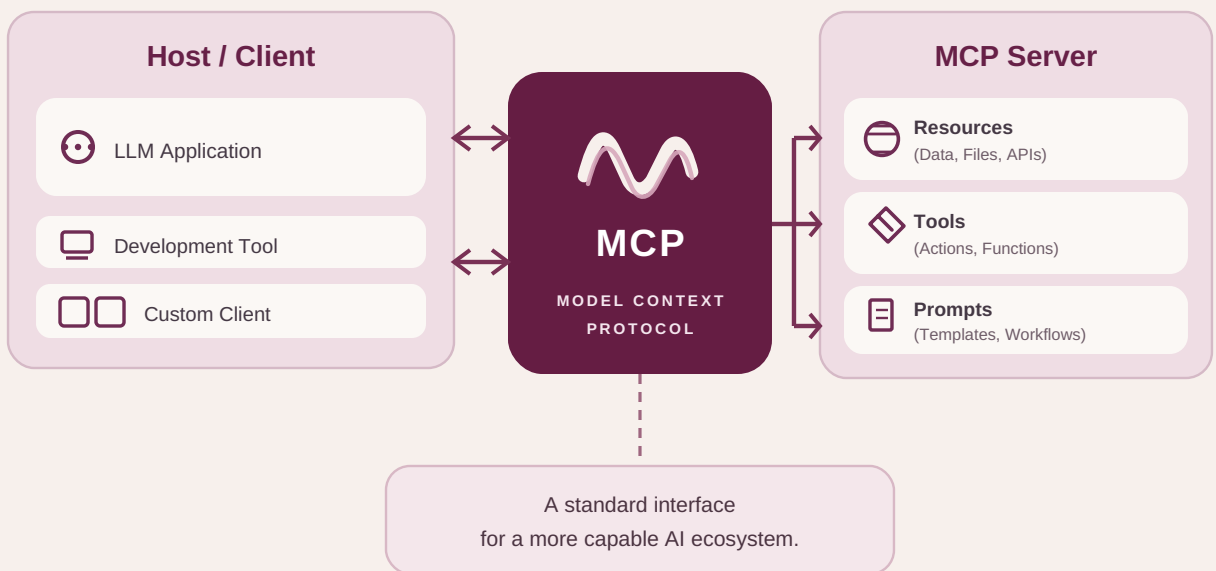


INTEGRATE



EXTEND

Host, client, and server roles; tools, resources, and prompts; and the JSON-RPC transport underneath it all.



Premium Revision Notes

CONCEPTS

Clear Explanations

IMPLEMENTATION

Practical Code

DIAGRAMS

Visual Learning

INTERVIEW PREP

Key Questions

The MCP Architecture, Primitives & Protocol

The Model Context Protocol connects two kinds of participants: a *host* that represents the AI application, and a *server* that exposes some capability the AI can use. Everything in MCP — the client layer, the three primitives, the JSON-RPC message format, and the transport mechanism — exists to make that one relationship reliable, decoupled, and scalable. This chapter builds the architecture from first principles: starting with the simplest possible two-party picture, refining it with the client layer, then going one level deeper into the data layer and transport layer that actually carry the conversation.

1. The Simplest Version of the Architecture

At its core, an MCP setup involves exactly two kinds of entities.

Host

The **host** is the AI application the user directly interacts with — a chat interface backed by an LLM (from any provider). It can be a ready-made application such as an AI desktop client or an IDE-integrated assistant, or a custom-built chatbot. Regardless of how it was built, the host's job is the same: take the user's prompt, forward it to an LLM, and act on the LLM's decisions.

Server

The **server** is a tool with the capability to execute a particular task. A server might wrap a code-hosting platform (so the host can manage repositories), a messaging platform (so the host can read or write channel messages), or a cloud storage account (so the host can manipulate files). A server's defining property is that it can do something the LLM cannot do on its own.

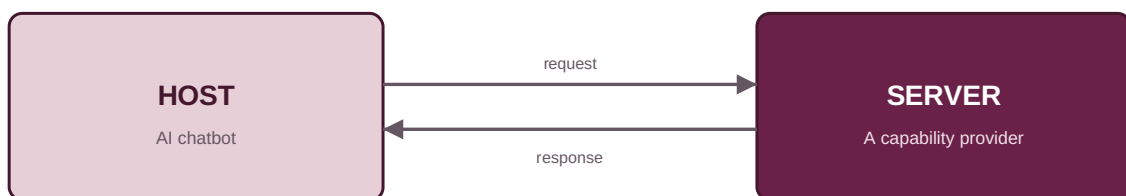


Figure 3.1 — In the simplest picture, all communication happens directly between the host and a server.

WALKING THROUGH A REQUEST

Suppose the user asks: *"Are there any new commits on the GitHub repo?"* The flow is:

1. The user sends the question to the host as a prompt.
2. The host forwards the prompt to the LLM as-is.
3. The LLM recognizes the answer isn't in its training data, so it needs an external tool.
4. The LLM checks which servers are available and identifies a GitHub server as relevant.
5. The LLM tells the host it doesn't know the answer, but the GitHub server can find it.
6. The host queries the GitHub server directly and receives the list of new commits.
7. The host passes this information back to the LLM, which now has enough context to answer.
8. The LLM produces the final answer, and the host displays it to the user.

IMPORTANT

This two-party picture is a simplification used only to introduce the concept. In a real MCP system, the host never talks to a server directly — that responsibility belongs to a client, introduced next.

🔄 Quick Revision

- The **host** is the AI application (chatbot) that talks to an LLM and to the user.
- The **server** is a capability provider — it can execute a specific task the LLM cannot do alone.
- A single host can, in principle, use many different servers (GitHub, Slack, Google Drive, and so on).
- The simplest picture treats host and server as talking directly — this is refined in the next section.

2. Introducing the Client

In reality, a host never communicates with a server directly. Whenever the host needs to talk to a server, it delegates that job to a helper entity: the **MCP client**. The client's defining skill is that it can speak the exact same MCP language the server speaks, which makes the low-level exchange straightforward.

The refined request flow

1. The user asks the host a question about the repository.
2. The host turns this into a high-level instruction, e.g. "find the recent commits in the repo," and hands it to the client.
3. The client converts this into an MCP-compatible request and sends it to the server.
4. Because the request is in MCP's standard format, the server can parse it immediately and execute the work.
5. The server returns a structured MCP response to the client.
6. The client translates that response into a form the host understands, and the host relays the answer to the user.

A strict one-to-one relationship

A client can be connected to exactly one server at a time. If a host needs to talk to a second server, it needs a second, separate client — and a third server requires a third client, and so on. A host with n connected servers therefore always has n clients running inside it, each dedicated to one server.

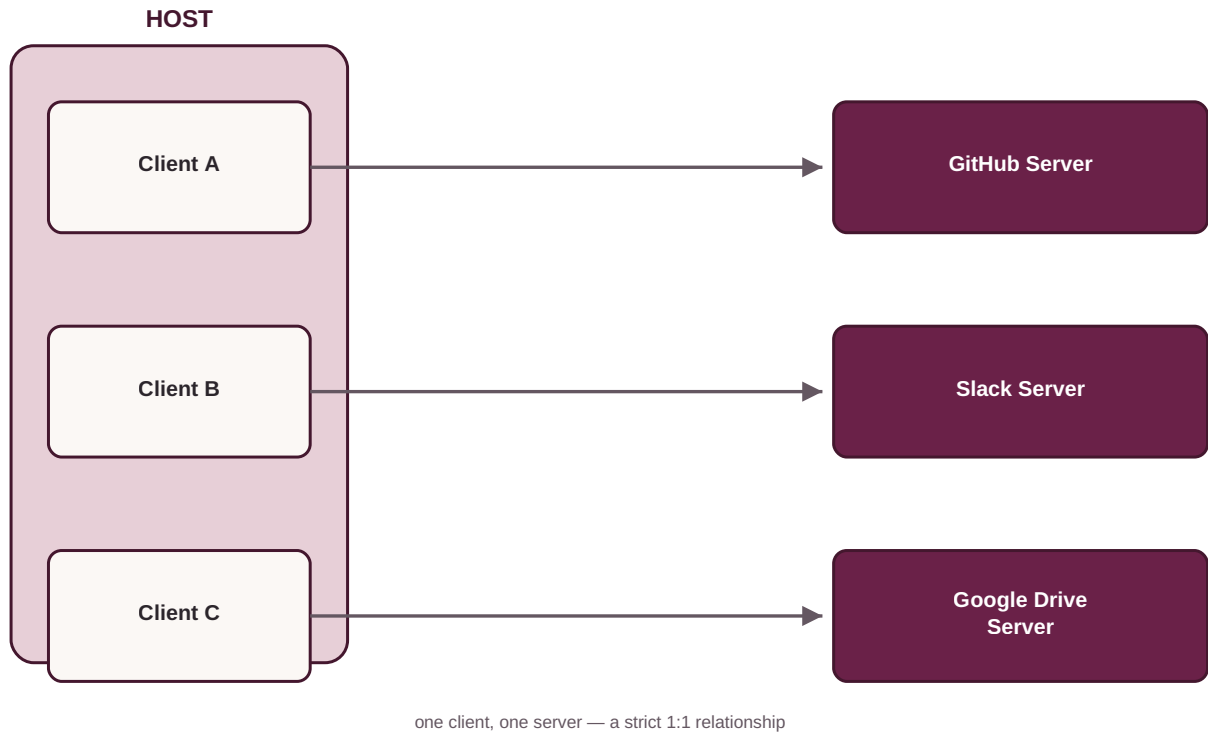


Figure 3.2 — Every server gets its own dedicated client; no two servers ever share a client.

REAL WORLD INSIGHT

A useful mental model: the host is like a phone, the client is like a SIM card, and the server is like a mobile network. The phone never talks to the network directly — the SIM mediates that connection, and each SIM is tied to exactly one network at a time. Wanting to reach multiple networks means installing multiple SIMs, exactly as connecting to multiple MCP servers means running multiple clients.

Why this architecture pays off

Benefit	What it means in practice
Decoupling	Each client-server communication channel is isolated. The host's connection to GitHub knows nothing about its connection to Slack. If one channel fails, the others keep working — a natural sense of safety in the system.
Scalability	Multiple client-server pairs can operate in parallel. If a request needs both a Slack action and a GitHub action, both clients can execute their calls at the same time rather than serially, keeping overall task execution fast.

BEST PRACTICE

When reasoning about scale, count clients, not servers: adding an n th server to a host always means adding an n th client. There is no shortcut where two servers share one client.

Quick Revision

- The host never talks to a server directly — every exchange goes through a dedicated **client**.
- Client ↔ server is strictly **one-to-one**: one client per server, one server per client.
- A host connected to n servers runs n clients internally.
- This design buys two things: **decoupling** (isolated failure domains) and **scalability** (parallel execution across servers).

3. MCP Primitives

Primitives are the things a server can offer to a host. Every MCP server exposes some combination of three primitive types.

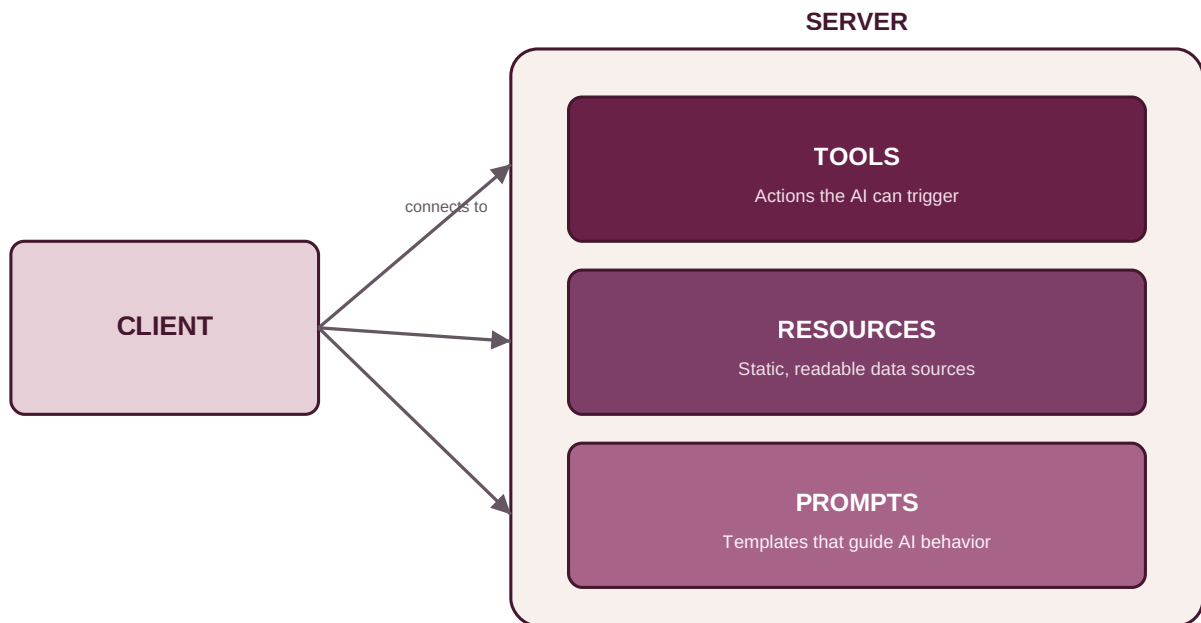


Figure 3.3 — A server offers up to three kinds of primitives to any client that connects to it.

Tools

Tools are actions the AI can ask the server to perform. A GitHub server, for example, might expose tools to count commits on a repository, list open issues, or report the number of repositories on an account. A Google Drive server might expose tools to search for a file or create a new one. Tools are the primitive used whenever something needs to be *done*.

Resources

Resources are structured, static data sources the AI can read. The distinguishing test is whether the underlying data changes on its own: a repository's README file is static content and therefore a resource, whereas the current commit count changes constantly and is better served by a tool. Similarly, a database schema fetched from a database server is a resource.

Prompts

Prompts are predefined prompt templates and instructions that a server offers to shape how the AI behaves when using that server. They solve a specific problem: without guidance, an LLM generating a request (say, a GitHub issue body) may produce something vague, because it was never told what a good request should contain.

WORKED EXAMPLE — THE PROMPT PRIMITIVE IN ACTION

Suppose the user says: *"Create an issue for a bug. The login button doesn't work."* Without a prompt template, the LLM might generate an issue with only a title and a one-line body — too vague to be useful. With a prompt primitive named something like `issue_report_prompt`, the server can instruct the LLM to always include a title, steps to reproduce, expected behavior, actual behavior, and environment details. The resulting issue is far more complete and directly usable.

WITHOUT A PROMPT TEMPLATE

```
Title: Login bug
Body: The login button doesn't work.
```

WITH THE `ISSUE_REPORT_PROMPT` TEMPLATE

```
Title: Bug - Login Button Not Working

**Steps to Reproduce:**
1. Open the login page
2. Enter valid credentials
3. Click the Login button

**Expected Behavior:**
User should be logged in and
redirected to dashboard.

**Actual Behavior:**
Nothing happens when clicking
the Login button.

**Environment:**
Chrome 121, macOS 14.2
```

COMMON MISTAKE

Treating prompts as mandatory for every server is a mistake. Prompt primitives are a specialized tool for guiding output quality in specific, recurring tasks — not every server needs one, and MCP does not require it.

Standard operations for each primitive

Alongside the three primitives, MCP defines standard operations for discovering and using them. These are the methods a client actually calls over the wire.

Primitive	Operation	Purpose
Tools	tools/list	Ask the server which tools it provides.
	tools/call	Run a specific tool with a given set of arguments.
Resources	resources/list	Ask which static resources are available.
	resources/read	Fetch the content of a specific resource.
	resources/subscribe	Ask to be notified when a resource changes.
	resources/unsubscribe	Stop receiving change notifications for a resource.
Prompts	prompts/list	Ask which prompt templates are available.
	prompts/get	Fetch a specific prompt template.

🔄 Quick Revision

- **Tools** = actions the AI can trigger; **Resources** = static data the AI can read; **Prompts** = templates that shape AI behavior.
- The static-vs-dynamic test decides tool vs resource: changing data needs a tool, unchanging data is a resource.
- Standard operations follow a `list / call-or-read` pattern per primitive, plus `subscribe/unsubscribe` for resources.
- Prompt primitives are optional and used only where output quality genuinely benefits from guidance.

4. The Data Layer: JSON-RPC 2.0

The **data layer** is the language and grammar the entire MCP ecosystem agrees on to communicate. Every request the client sends and every response the server returns is written according to one specification: **JSON-RPC 2.0**.

What is RPC?

A **Remote Procedure Call (RPC)** allows a program to execute a function on another computer as if it were local, hiding the details of network communication and data transfer. This abstraction is what makes distributed applications tractable: instead of writing `add(2, 3)` against a function that happens to live on another machine, a program sends a request such as "please run `add` with parameters 2 and 3," and the remote machine executes it and returns the result.

JSON-RPC = RPC + JSON

JSON-RPC combines the concept of remote procedure calls with the simplicity of JSON, letting developers structure RPC requests and responses in a standardized JSON format. The version MCP uses is 2.0, hence "JSON-RPC 2.0."

REQUEST STRUCTURE

```
{
  "jsonrpc": "2.0",
  "method": "add",
  "params": [2, 3],
  "id": 1
}
```

The `jsonrpc` field pins the protocol version. `method` names the remote function to execute. `params` carries its arguments. `id` is a correlation token: the response echoes back the same `id` so the client knows exactly which request it belongs to.

RESPONSE STRUCTURE

SUCCESS

```
{
  "jsonrpc": "2.0",
  "result": 5,
  "id": 1
}
```

ERROR

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32601,
    "message": "Method not found"
  },
  "id": 1
}
```

Error responses carry a standardized integer `code` (comparable in spirit to HTTP status codes) alongside a human-readable `message`, and still echo the original `id` so the client can match the failure to its request.

JSON-RPC inside MCP

Every standard operation introduced in Section 3 is, under the hood, a JSON-RPC call. Discovering a server's tools looks like this:

REQUEST — TOOLS/LIST

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list"
}
```

RESPONSE

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      { "name": "listIssues" },
      { "name": "listPulls" }
    ]
  }
}
```

Calling a specific tool adds a `params` object carrying the tool name and its arguments:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "listIssues",
    "arguments": { "owner": "octocat", "repo": "hello-world" }
  }
}
```

IMPORTANT

The exact set of arguments a tool needs is discoverable — the `tools/list` response includes each tool's expected parameters, so a client always knows what to pass into `tools/call` before making the call.

Notifications: fire-and-forget messages

A normal request-response exchange obligates the receiver to reply. A **notification** does not: either side can send one, and the receiver has no responsibility to respond. Notifications are used, for example, when a client has subscribed to a resource and the server needs to announce that the resource changed.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/resource_updated",
  "params": {
    "uri": "file:///README.md",
    "updatedBy": "server",
    "updatedAt": "2025-01-14T10:32:00Z"
  }
}
```

IMPORTANT

Notice the notification carries no `id` field. An `id` exists specifically so a response can be matched back to its request — since no response is expected for a notification, there is nothing to match, so the field is omitted.

Batching

JSON-RPC also allows multiple requests to be sent together as a single batch, wrapped in a JSON array. A client that needs both the issue list and the pull request list can send both calls in one round trip, and the server returns both results merged into a single response.

🔄 Quick Revision

- The **data layer** is the shared grammar for every MCP message; it is built on **JSON-RPC 2.0**.
- An RPC lets a program call a function on another machine as if it were local.
- A request always carries `jsonrpc`, `method`, `params`, and `id`; a response carries either `result` or `error`, plus the same `id`.
- **Notifications** are fire-and-forget messages with no `id` and no expected response.
- JSON-RPC supports **batching** — multiple calls sent together in one array.

5. Why JSON-RPC Instead of REST?

MCP's designers could have used REST APIs to move data between hosts and servers — a familiar, well-understood option. JSON-RPC was chosen instead, for five main reasons.

#	Reason	Why it matters for MCP
1	Lightweight	A JSON-RPC message carries no headers or transport metadata the way an HTTP request does, so it is faster to construct, easier to debug, and cheaper to transmit.
2	Bidirectional by design	REST is inherently one-directional (client requests, server responds). JSON-RPC allows either side to initiate a request, which MCP relies on for advanced flows.
3	Transport-agnostic	REST is tied to HTTP. JSON-RPC does not assume any particular transport — it can run over STDIO, HTTP, WebSockets, or a custom transport, without changing the message format.
4	Native batching	Multiple calls can be sent in a single request, useful whenever an AI workflow needs several server operations at once.
5	Native notifications	REST has no built-in concept of a message that expects no reply. JSON-RPC does, which MCP uses for change notifications and similar signals.

INTERVIEW TIP

If asked "why not just use REST for MCP," the strongest answer centers on transport independence: MCP had to support both local processes (no network at all) and networked servers with a single message format, and only a transport-agnostic protocol like JSON-RPC can do that cleanly.

6. The Transport Layer

The **transport layer** is the mechanism that physically moves JSON-RPC messages between client and server. Which transport is used depends entirely on where the server lives.

Two types of MCP servers

Server type	Definition	Transport used
Local server	Runs on the same machine as the host.	STDIO
Remote server	Runs on a different machine, reachable over a network or the internet.	HTTP + SSE

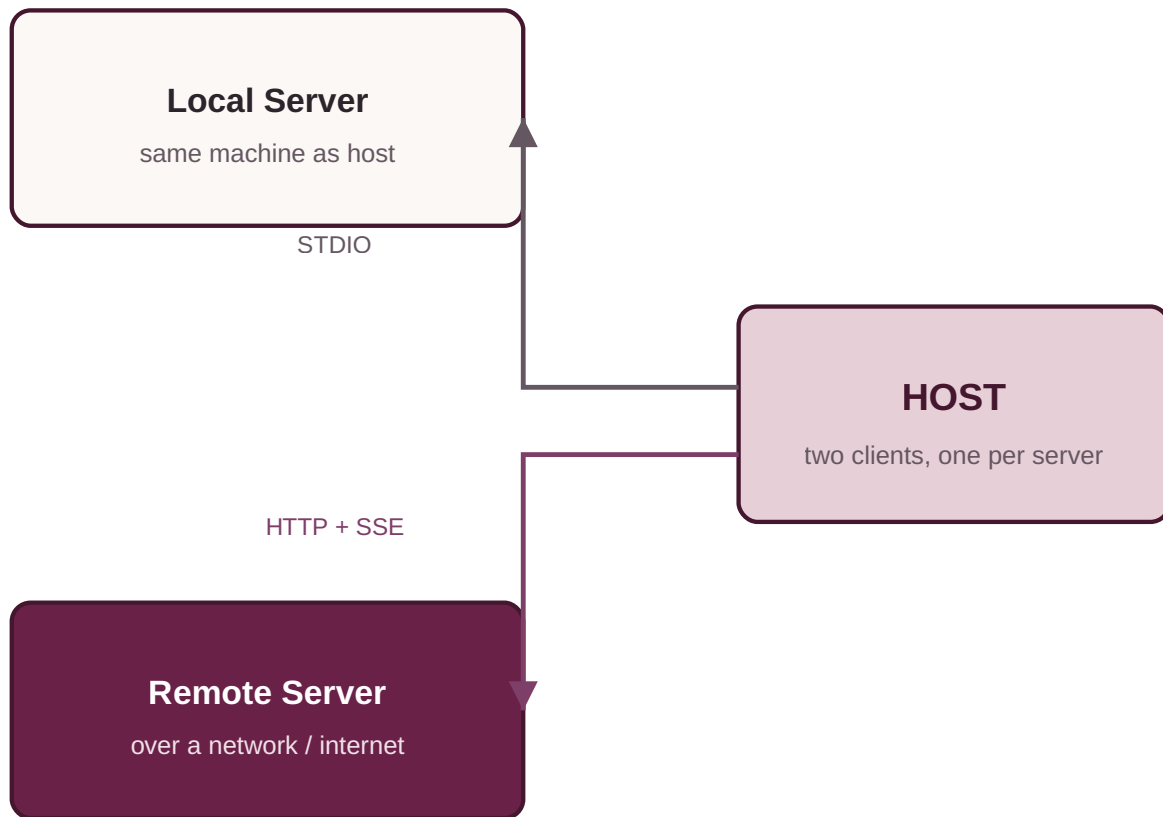


Figure 3.4 — A host can hold simultaneous connections of both kinds: *STDIO* to a local server, *HTTP + SSE* to a remote one.

Local servers: **STDIO** transport

STDIO (Standard Input/Output) refers to the built-in input and output streams every running program has. For a local server, MCP uses this mechanism as the transport layer between client and server, in three steps:

1. The host launches the server as a **sub-process** on the same machine, which establishes a parent-child relationship between them.
2. Because of that relationship, the host's client gains control of the server's standard input and standard output.
3. The client writes JSON-RPC messages into the server's standard input; the server processes them and writes its response back through standard output, which the host reads.

BEST PRACTICE

A useful sanity check for STDIO: running `python3 hello.py` from a terminal and typing input at a prompt is the same underlying mechanism — the terminal (host) launches the program (server) as a sub-process and exchanges data with it over standard input/output.

BENEFITS OF STDIO TRANSPORT

- **Fast** — both processes run on the same machine, so message exchange has no network latency.
- **Secure** — no network ports are opened, which removes an entire class of network-based attacks.
- **Simple to implement** — virtually every programming language already supports standard input/output natively.

Remote servers: HTTP + SSE transport

For remote servers, MCP uses **HTTP** as the base transport, because it is the most widely supported application protocol on the internet — a host can reach a remote server anywhere using it. The host sends its JSON-RPC message as the JSON payload of an HTTP **POST** request, which also means every standard HTTP authentication method (such as API keys) is available for free.

The new piece is **SSE (Server-Sent Events)**, an extension of HTTP used for streaming. Instead of returning one large JSON blob, the server can send multiple messages to the client over a single open connection, streaming the response chunk by chunk as it becomes available. SSE is the right choice whenever a task is long-running or the client needs incremental progress updates — both common needs in AI workflows.

REAL WORLD INSIGHT

Streaming responses aren't just a UX nicety here — for a multi-step agent task that may run for tens of seconds, SSE lets the host surface intermediate progress to the user instead of leaving them staring at a blank screen until the entire task finishes.

Why transport-agnostic JSON-RPC matters here

This is the payoff of Section 5's third reason. Local servers need STDIO; remote servers need HTTP + SSE — two completely different transports. But the message format riding on top of both is identical: JSON-RPC 2.0. Had MCP been built on REST instead, the local-server case would have been awkward, since REST assumes HTTP and a local process would have needed its own lightweight web server just to receive requests. Separating the data layer from the transport layer means the transport can change (today STDIO and HTTP+SSE, potentially WebSockets tomorrow) without requiring any change to how messages are written.

🔄 Quick Revision

- The **transport layer** physically moves JSON-RPC messages between client and server.
- **Local servers** run on the host's own machine and use **STDIO**, established via a parent-child sub-process relationship.
- **Remote servers** run elsewhere and use **HTTP + SSE**; HTTP carries the request as a POST payload, SSE streams the response.
- Because JSON-RPC is transport-agnostic, both server types share exactly the same message format — only the transport differs.

Key Takeaways

- The **host** is the AI application; the **server** provides a capability; the **client** is the mandatory intermediary, connected one-to-one with a single server.
- A host connected to n servers runs n clients, giving the architecture both decoupling and scalability.
- Servers expose up to three **primitives**: tools (actions), resources (static data), and prompts (behavior-shaping templates) — each with its own standard **list/call-or-read** operations.
- The **data layer** standardizes every message as **JSON-RPC 2.0**: requests carry **method/params/id**, responses carry **result** or **error**, and notifications skip the **id** entirely.
- JSON-RPC was chosen over REST for being lightweight, bidirectional, transport-agnostic, batchable, and notification-capable.
- The **transport layer** is separate from the data layer: local servers use **STDIO**, remote servers use **HTTP + SSE**, and the same JSON-RPC messages travel over either.

Revision Sheet

A one-line reminder for every core term introduced in this chapter.

Host

The AI application / chatbot that talks to the LLM and the user; never talks to a server directly.

Client

Mediates all communication with exactly one server; speaks the same MCP language the server speaks.

Server

Exposes a capability via tools, resources, and/or prompts; local (STDIO) or remote (HTTP+SSE).

Tools

Actions the AI can trigger on the server, e.g. creating an issue or searching a drive.

Resources

Static, readable data on the server, e.g. a README file or a database schema.

Prompts

Predefined templates that guide how the LLM should phrase requests to that server.

JSON-RPC 2.0

The message format for the data layer: `method/params/id` requests, `result/error` responses.

Notification

A JSON-RPC message with no `id` and no expected reply — fire and forget.

STDIO transport

Used for local servers; the host launches the server as a sub-process and exchanges data over standard input/output.

HTTP + SSE transport

Used for remote servers; requests travel as HTTP POST payloads, responses can stream via Server-Sent Events.

Interview Questions

- Q1.** Why can't a host communicate with an MCP server directly, and what role does the client play instead?
- Q2.** Explain the one-to-one relationship between clients and servers, and what it implies for a host connected to five different servers.
- Q3.** What is the difference between a tool and a resource? Give an example of each.
- Q4.** Why does MCP need a prompt primitive if tools and resources already exist?
- Q5.** What fields make up a JSON-RPC 2.0 request, and what does each one do?
- Q6.** How does a notification differ from a normal JSON-RPC request, and why does it omit the `id` field?
- Q7.** Give three reasons MCP uses JSON-RPC 2.0 instead of a REST API for its data layer.
- Q8.** What transport does MCP use for a local server, and how is the connection established?
- Q9.** Why is SSE useful for remote MCP servers, and what kind of tasks benefit most from it?
- Q10.** Explain why separating the data layer from the transport layer is considered one of MCP's most important design decisions.

Further Reading

- Model Context Protocol specification — architecture and lifecycle sections
- JSON-RPC 2.0 specification
- Server-Sent Events (SSE) — MDN Web Docs